

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code: DSA05

CO Assessed: CO2

Weightage:

Course Title: Query Processing for Data Science With Real Time Applications

Assessment: Tool 2 – Schema Analysis and Viva Voce

Total Marks: 35 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)

Student Name: Sandra Sudevan

Section / Batch: 2024

Faculty In-charge: Dr. B.SHYAMALA GOWRI

Reg. No.: 192424422

Date of Submission: 04/08/26

Project Mode: Individual Submission

Code of Conduct

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sandra Sudevan

Identification of Entities and Attributes (5 Marks)	Understanding of Relationships (5 Marks)	Analysis of Keys and Constraints (5 Marks)	Detection of Design Issues (10 Marks)	Viva Voce (5 Marks)	Total (35 Marks)

5. Banking System

Tables:

- Customer(CustomerID, Name)
- Account(AccountNo, CustomerID, Balance)
- Transaction(TransactionID, AccountNo, Amount, Type)

One customer can have multiple accounts.

Procedure

1. Open VS Code and create banking_system.sql.
2. Connect to MySQL through the VS Code terminal.
3. Create and select BankingDB.
4. Create Customer, Account, and Transaction tables.
5. Define primary and foreign key constraints.
6. Insert customer, account, and transaction records.
7. Use SELECT statements to display the data.
8. Use JOIN to display complete customer, account, and transaction information.
9. Verify that one customer can have multiple accounts.

Viva Questions

Q1. Why should transactions be stored separately from account details?

Transactions should be stored separately because an account can have many transactions over time.

- Account stores current/static information like AccountNo, CustomerID, Balance.
- Transaction stores individual activities like deposit, withdrawal, transfer, and amount.
- It avoids repeating account details for every transaction.
- It maintains a proper transaction history.
- It makes the database easier to maintain and query.

Q2. How would you redesign the schema to support joint accounts?

Currently, Account(CustomerID, AccountNo, Balance) allows only one customer per account.

For joint accounts, create a many-to-many relationship using a separate table.

- Customer(CustomerID, Name)
- Account(AccountNo, Balance)
- AccountHolder(AccountNo, CustomerID)

```
mysql> DESC Customer;
```

Field	Type	Null	Key	Default	Extra
CustomerID	int	NO	PRI	NULL	
CustomerName	varchar(50)	NO		NULL	
AccountType	varchar(20)	YES		NULL	
Balance	decimal(10,2)	YES		NULL	
Branch	varchar(50)	YES		NULL	

```
5 rows in set (1.674 sec)
```

```
mysql> DESC Account;
```

Field	Type	Null	Key	Default	Extra
AccountNo	int	NO	PRI	NULL	
CustomerID	int	YES	MUL	NULL	
Balance	decimal(10,2)	YES		0.00	

```
3 rows in set (0.240 sec)
```

```
mysql> DESC `Transaction`;
```

Field	Type	Null	Key	Default	Extra
TransactionID	int	NO	PRI	NULL	
AccountNo	int	YES	MUL	NULL	
Amount	decimal(10,2)	YES		NULL	
Type	varchar(20)	YES		NULL	

```
4 rows in set (0.246 sec)
```

```
mysql> SELECT * FROM Customer;
```

CustomerID	CustomerName	AccountType	Balance	Branch
101	Rahul	Savings	25000.00	Chennai
102	Priya	Current	50000.00	Coimbatore
103	Arun	Savings	15000.00	Madurai
104	Sneha	Current	80000.00	Salem

```
4 rows in set (0.018 sec)
```

```
mysql> SELECT * FROM Account;
```

AccountNo	CustomerID	Balance
10001	101	25000.00
10002	101	15000.00
10003	102	30000.00
10004	103	12000.00

```
4 rows in set (0.009 sec)
```

```
mysql> SELECT * FROM `Transaction`;
```

TransactionID	AccountNo	Amount	Type
1	10001	5000.00	Deposit
2	10001	2000.00	Withdrawal
3	10002	3000.00	Deposit
4	10003	5000.00	Withdrawal

```
4 rows in set (0.017 sec)
```

```
ERROR 1054 (42S22): Unknown column 'C.Name' in 'field list'
```

```
mysql> DESC Customer;
```

Field	Type	Null	Key	Default	Extra
CustomerID	int	NO	PRI	NULL	
CustomerName	varchar(50)	NO		NULL	
AccountType	varchar(20)	YES		NULL	
Balance	decimal(10,2)	YES		NULL	
Branch	varchar(50)	YES		NULL	

```
5 rows in set (0.585 sec)
```

```
ON C.CustomerID = A.CustomerID;
```

CustomerID	CustomerName	AccountNo	Balance
101	Rahul	10001	25000.00
101	Rahul	10002	15000.00
102	Priya	10003	30000.00
103	Arun	10004	12000.00

```
4 rows in set (0.041 sec)
```

```

> ON a.AccountNo = c.AccountNo,
+-----+-----+-----+-----+-----+
| CustomerName | AccountNo | TransactionID | Amount | Type |
+-----+-----+-----+-----+-----+
| Rahul        | 10001     | 1             | 5000.00 | Deposit |
| Rahul        | 10001     | 2             | 2000.00 | Withdrawal |
| Rahul        | 10002     | 3             | 3000.00 | Deposit |
| Priya        | 10003     | 4             | 5000.00 | Withdrawal |
+-----+-----+-----+-----+-----+
4 rows in set (0.071 sec)

```

```
mysql> DESC Customer;
```

```

+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| CustomerID | int           | NO   | PRI | NULL    |       |
| Name       | varchar(50)   | NO   |     | NULL    |       |
| AccountType | varchar(20)   | YES  |     | NULL    |       |
| Balance    | decimal(10,2) | YES  |     | NULL    |       |
| Branch     | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.229 sec)

```

```
mysql> DESC Account;
```

```

+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| AccountNo  | int           | NO   | PRI | NULL    |       |
| CustomerID | int           | YES  | MUL | NULL    |       |
| Balance    | decimal(10,2) | YES  |     | 0.00    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.055 sec)

```

```
mysql> DESC `Transaction`;
```

```

+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| TransactionID | int           | NO   | PRI | NULL    |       |
| AccountNo    | int           | YES  | MUL | NULL    |       |
| Amount       | decimal(10,2) | YES  |     | NULL    |       |
| Type        | varchar(20)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.037 sec)

```

6. Employee Payroll System

Tables:

- Employee(EmployeeID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Payroll(PayrollID, EmployeeID, Salary, Month)

Employees belong to departments and receive monthly salaries.

Procedure

1. Open MySQL and select/create a database for the Employee Payroll System.
2. Create the Department table with DepartmentID as the primary key and DepartmentName to store department details.
3. Create the Employee table with EmployeeID as the primary key and DepartmentID as a foreign key referencing the Department table.
4. Create the Payroll table with PayrollID as the primary key and EmployeeID as a foreign key referencing the Employee table.
5. Insert department records into the Department table.
6. Insert employee records into the Employee table and assign each employee to a department using DepartmentID.
7. Insert payroll records containing the employee's salary and corresponding month.
8. Use SQL JOIN queries to combine Employee, Department, and Payroll information and display complete payroll details.
9. Verify the data using SELECT queries and check that the primary key and foreign key constraints work correctly.
10. Execute the required viva queries to demonstrate how separating department information reduces redundancy and maintains data consistency.

```
mysql> SELECT * FROM Department;
```

DepartmentID	DepartmentName
1	HR
2	IT
3	Finance

```
3 rows in set (0.110 sec)
```

```
mysql> SELECT * FROM Employee;
```

EmployeeID	Name	DepartmentID
101	Ravi	1
102	Anu	2
103	Kumar	2
104	Priya	3

```
4 rows in set (0.018 sec)
```

```
mysql> SELECT * FROM Payroll;
```

PayrollID	EmployeeID	Salary	Month
1001	101	30000.00	January
1002	102	45000.00	January
1003	103	40000.00	January
1004	104	35000.00	January

```
4 rows in set (0.195 sec)
```

```
mysql> SELECT
->     e.EmployeeID,
->     e.Name,
->     d.DepartmentName,
->     p.Salary,
->     p.Month
-> FROM Employee e
-> JOIN Department d
-> ON e.DepartmentID = d.DepartmentID
-> JOIN Payroll p
-> ON e.EmployeeID = p.EmployeeID;
```

EmployeeID	Name	DepartmentName	Salary	Month
101	Ravi	HR	30000.00	January
102	Anu	IT	45000.00	January
103	Kumar	IT	40000.00	January
104	Priya	Finance	35000.00	January

4 rows in set (0.406 sec)

Viva Questions

Q1. Why is Department information separated into a different table?

Department information is separated into a different table to avoid duplication. Multiple employees can belong to the same department, so we store the department details once and use DepartmentID as a foreign key in the Employee table.

Q2. What redundancy issues would arise if department details were stored in Employee records?

Storing department details in the Employee table causes redundancy because the same department information is repeated for multiple employees. It can also cause update anomalies and data inconsistency when department information changes. Separating the Department table avoids these problems.